HCL TECH TRAINING PRACTICE

E-Commerce Sales Analysis

SQL practice report using Google BigQuery

Prepared by	Naveen Kumar S
Programme	B.Tech, Information Science and Engineering
Institution	Jain (Deemed-to-be University), Bengaluru
Platform	GCP (Google Cloud Platform): BigQuery
Dataset	ecom.orders
Date	30 October 2026

1. Objective

An e-commerce company receives about 1,000 orders every day. The business team wants to identify its highest-revenue products, its top customers and its monthly sales. This report solves ten business tasks with SQL in Google BigQuery: cleaning the data, calculating revenue, ranking products and analysing customer behaviour.

SUMMARY OF RESULTS (SAMPLE DATASET)

Task	Business question	Result
1	Remove duplicate orders	`orders_clean` created with one row per `order_id`
2	Net revenue	58,500
3	Top 3 products per region	South: Laptop, Mouse; North: Phone; East: Headphones
4	Inactive customers	C1, C2 and C3 (no delivered order in the last 90 days as of 30 Oct 2026)
5	Customers buying 3 months in a row	None in the sample data
6	Customer lifetime value	C1 highest at 46,000
7	Monthly sales	Jan 54,000; Feb 1,000; Mar 3,500
8	Unit price below 10,000 (March 2026)	Orders 3 and 6
9	Repeat purchases of one product in 6 months	None in the sample data
10	Product sales for a month (January 2026)	Laptop 45,000; Phone 9,000

2. Dataset

The analysis uses one table, `ecom.orders`, with the following structure.

Column	Type	Description
order_id	INT64	Unique order number
customer_id	STRING	Customer identifier
order_date	DATE	Date the order was placed
product_id	STRING	Product code
product	STRING	Product name
quantity	INT64	Units ordered
unit_price	NUMERIC	Price per unit
region	STRING	Sales region
status	STRING	Delivered or Cancelled

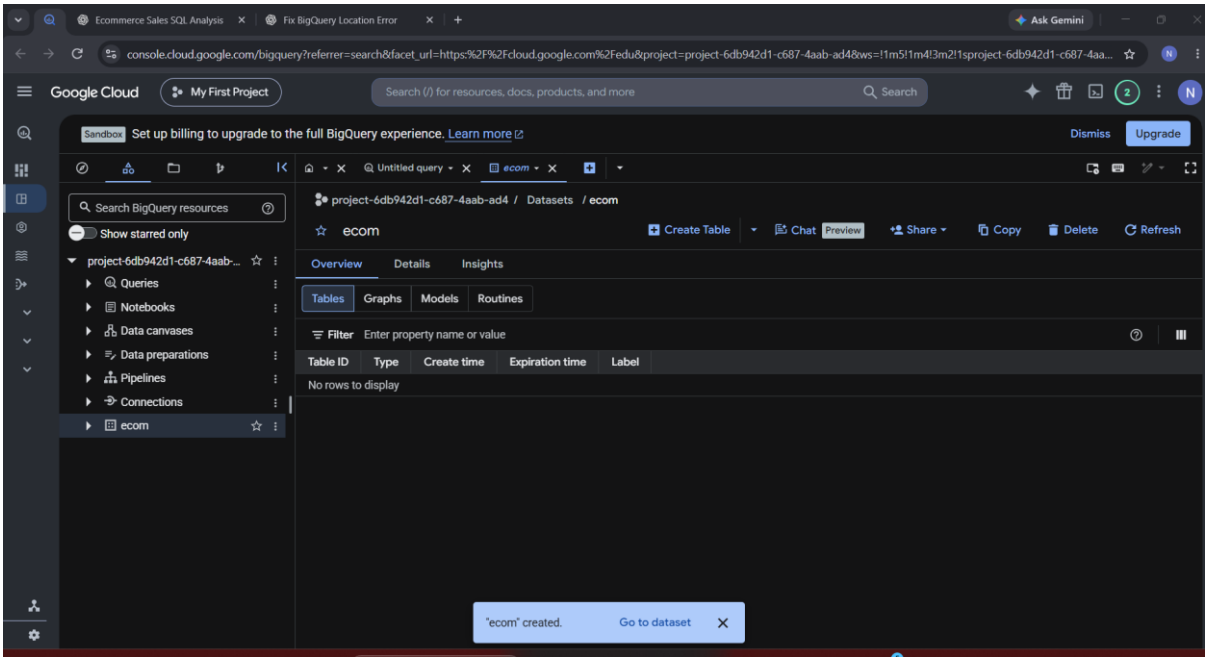
SAMPLE RECORDS USED FOR TESTING

Order	Customer	Date	Product	Qty	Unit price	Region	Status
1	C1	2026-01-10	Laptop	1	45,000	South	Delivered
2	C1	2026-02-12	Mouse	2	500	South	Delivered
3	C2	2026-03-15	Mouse	1	500	South	Delivered
4	C2	2026-01-20	Phone	1	9,000	North	Delivered
5	C2	2026-02-05	Laptop	1	45,000	North	Cancelled
6	C3	2026-03-18	Headphones	1	3,000	East	Delivered

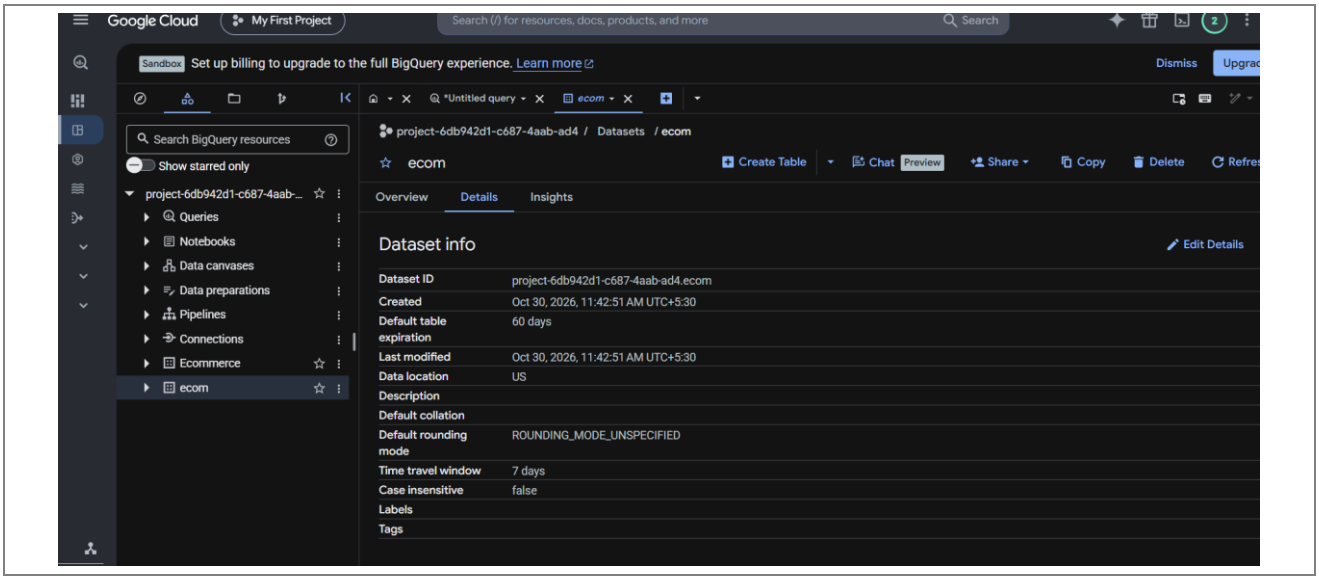
Revenue rule: revenue = `quantity` × `unit_price`, counted only for orders with status `DELIVERED`. Cancelled orders are excluded from every revenue figure.

3. Environment Setup and Table Creation

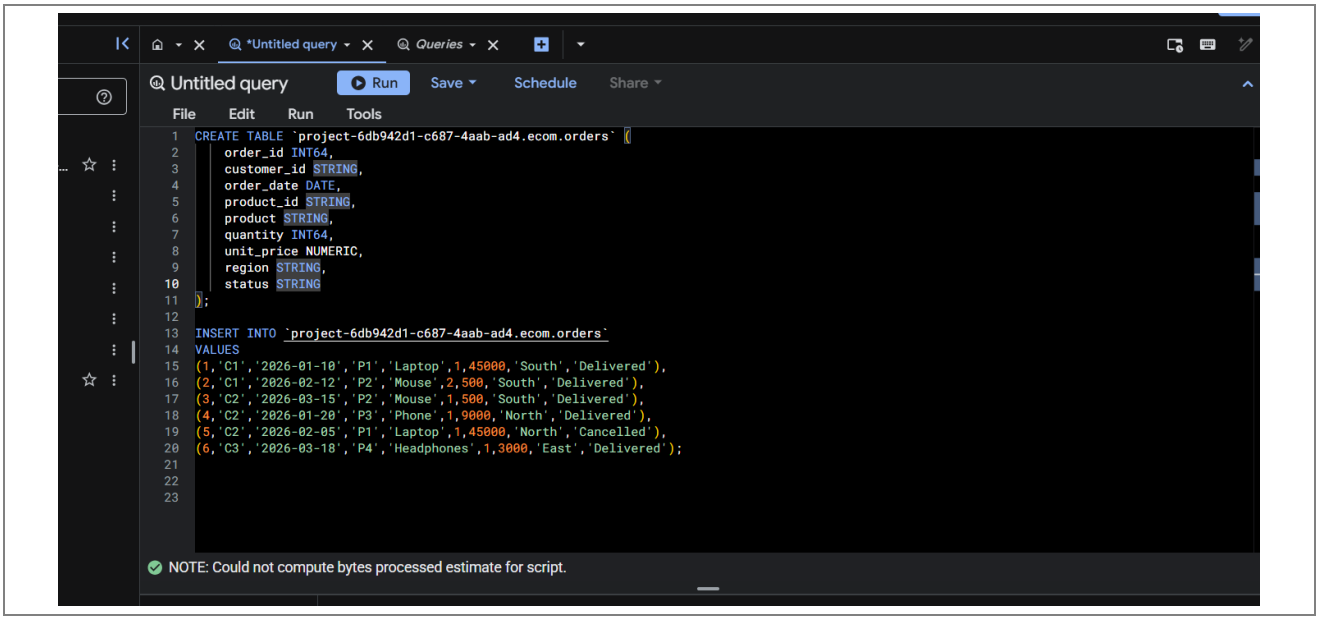
A BigQuery dataset named `ecom` was created in the project. The `orders` table was then created and loaded with the six sample records using one script.



Screenshot: dataset `ecom` created in the BigQuery project.



Screenshot: dataset details (data location US, time travel window 7 days).



Screenshot: script that creates the `orders` table and inserts the six sample records.

4. Task Solutions

Solutions prepared by Naveen Kumar S.

Task 1: Remove duplicate orders

Approach: number the rows within each `order_id` with `ROW_NUMBER()` and keep only row 1. The result is stored in a clean table, `orders_clean`, which every later task uses.

QUERY

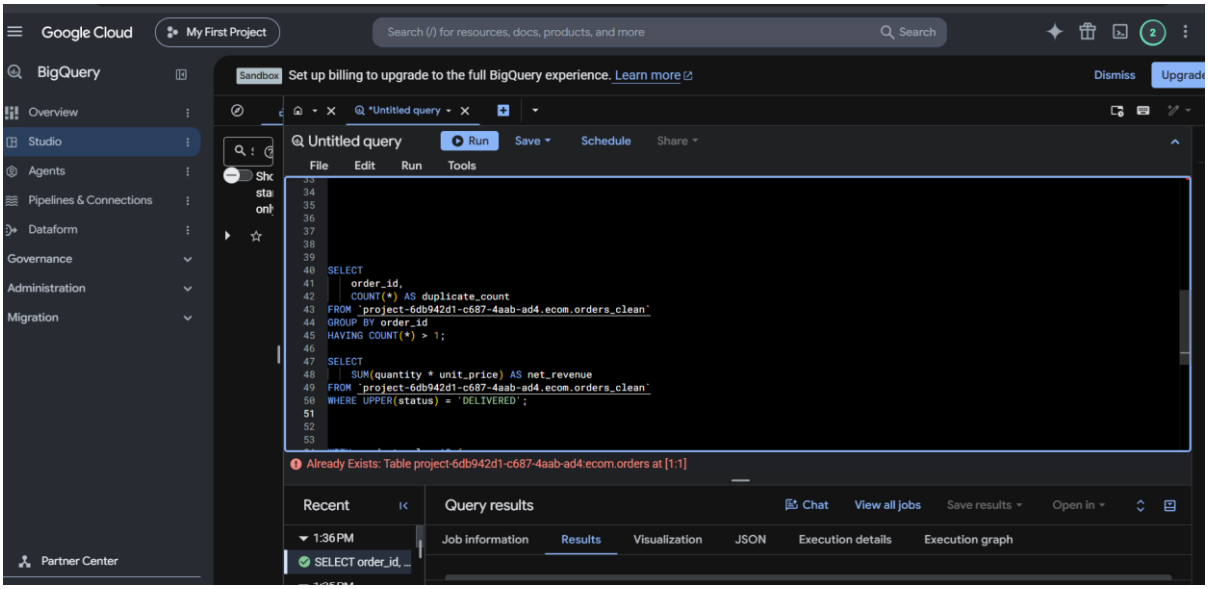
```sql
CREATE OR REPLACE TABLE ecom.orders_clean AS
SELECT * EXCEPT (rn)
FROM (
 SELECT *,
 ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY order_date DESC) AS rn
 FROM ecom.orders
)
WHERE rn = 1;
```

Screenshot: query that removes duplicate orders and creates `orders_clean`.

Verification: the duplicate check (`GROUP BY order_id HAVING COUNT(*) > 1`) is shown in the screenshot under Task 2 and should return no rows.

Task 2: Net revenue

Approach: sum `quantity` × `unit_price` for delivered orders only.

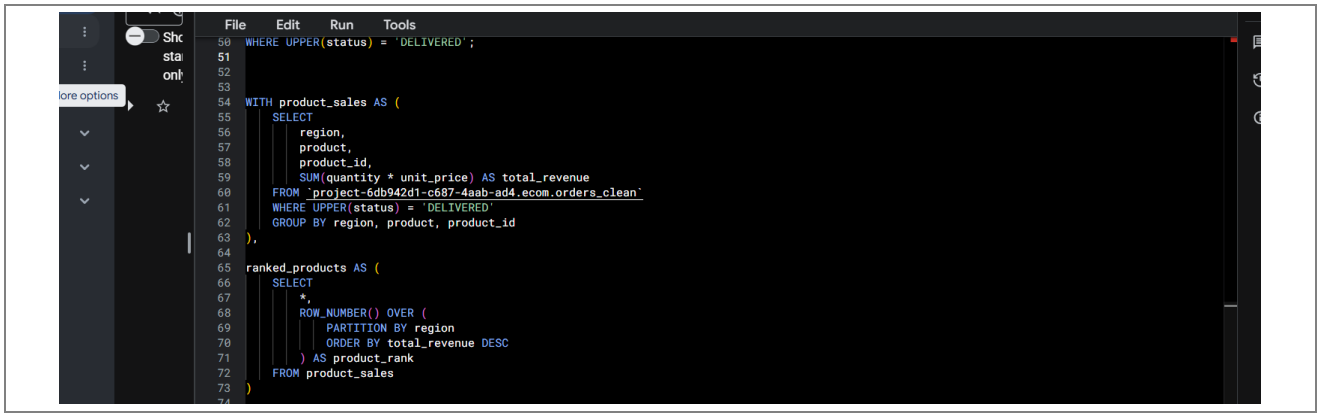


Screenshot: duplicate check on `orders_clean` and the net revenue query.

Result: net revenue = **58,500**. The cancelled laptop order (45,000) is excluded.

Task 3: Top 3 products in each region

Approach: calculate revenue per product per region in a CTE, rank the products with `ROW_NUMBER()` partitioned by region, and keep ranks 1 to 3.



Screenshot: `product_sales` and `ranked_products` CTEs.



Screenshot: final SELECT keeping `product_rank <= 3`, ordered by region.

RESULT

Region	Product	Total revenue	Rank
East	Headphones	3,000	1
North	Phone	9,000	1
South	Laptop	45,000	1
South	Mouse	1,500	2

Task 4: Identify inactive customers

Approach: a customer is inactive when the latest delivered order is more than 90 days old. The threshold can be changed to match the business definition.

QUERY



Screenshot: inactive customers query (no delivered order in the last 90 days).

Task 5: Customers who purchased for 3 continuous months

Approach: take each customer's distinct purchase months and number them. Subtracting that number (in months) from the month gives the same value for consecutive months, so a group of three or more rows is a streak.

QUERY

```sql
WITH monthly AS (
 SELECT DISTINCT customer_id,
 DATE_TRUNC(order_date, MONTH) AS month
 FROM ecom.orders_clean
 WHERE UPPER(status) = 'DELIVERED'
),
grouped AS (
 SELECT customer_id, month,
 DATE_SUB(month,
 INTERVAL ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY month) MONTH
 ) AS grp
 FROM monthly
)
SELECT customer_id,
 MIN(month) AS streak_start,
 MAX(month) AS streak_end,
 COUNT(*) AS months_in_streak
FROM grouped
GROUP BY customer_id, grp
HAVING COUNT(*) >= 3;
```

Screenshot: query to find customers who purchased in 3 continuous months.

Result: no customer has a 3-month streak in the small sample (C1 bought in January and February; C2 in January and March). On the full dataset the query lists the loyal customers.

Task 6: Customer lifetime value (CLV) report

Approach: for each customer, calculate total orders, lifetime revenue, average order value, first and last order dates and the active period in days.

QUERY

```sql
SELECT customer_id,
 COUNT(DISTINCT order_id) AS total_orders,
 SUM(quantity * unit_price) AS lifetime_value,
 ROUND(SAFE_DIVIDE(SUM(quantity * unit_price),
 COUNT(DISTINCT order_id)), 2) AS avg_order_value,
 MIN(order_date) AS first_order,
 MAX(order_date) AS last_order,
 DATE_DIFF(MAX(order_date), MIN(order_date), DAY) AS customer_lifespan_days
FROM ecom.orders_clean
WHERE UPPER(status) = 'DELIVERED'
GROUP BY customer_id
ORDER BY lifetime_value DESC;
```

Screenshot: customer lifetime value (CLV) query.

RESULT

Customer	Total orders	Lifetime value	Avg order value
C1	2	46,000	23,000
C2	2	9,500	4,750
C3	1	3,000	3,000

C1 is the top customer by lifetime value.

Task 7: Monthly sales

Approach: group delivered orders by month.

QUERY

```
SELECT DATE_TRUNC(order_date, MONTH) AS sales_month,
       COUNT(DISTINCT order_id) AS total_orders,
       SUM(quantity * unit_price) AS monthly_sales
FROM ecom.orders_clean
WHERE UPPER(status) = 'DELIVERED'
GROUP BY sales_month
ORDER BY sales_month;
```

RESULT

Month	Total orders	Monthly sales
2026-01	2	54,000
2026-02	1	1,000
2026-03	2	3,500

The monthly figures add up to 58,500, which matches the net revenue in Task 2.

Task 8: Unit price below 10,000 in a particular month

Approach: filter on `unit_price` and the date range of the chosen month (March 2026 shown).

QUERY

```
SELECT order_id, customer_id, order_date, product, unit_price, quantity
FROM ecom.orders_clean
WHERE unit_price < 10000
  AND order_date BETWEEN '2026-03-01' AND '2026-03-31'
ORDER BY unit_price DESC;
```

Result: order 6 (Headphones, 3,000) and order 3 (Mouse, 500).

Task 9: Customers who bought a product multiple times in 6 months

Approach: filter one product and a 6-month window, group by customer and keep customers with more than one order (product P2, January to June 2026 shown).

QUERY

```
SELECT customer_id, product_id, product,
       COUNT(*) AS times_purchased,
       SUM(quantity) AS total_units
FROM ecom.orders_clean
WHERE product_id = 'P2'
  AND order_date BETWEEN '2026-01-01' AND '2026-06-30'
  AND UPPER(status) = 'DELIVERED'
GROUP BY customer_id, product_id, product
HAVING COUNT(*) > 1;
```

Result: the small sample has no repeat buyer of P2 (C1 and C2 bought once each), so no rows are returned. For a rolling window, replace the date filter with `order_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 6 MONTH)`.

Task 10: Total sales for each product in a particular month

Approach: filter the month (January 2026 shown) and total units and revenue per product.

QUERY

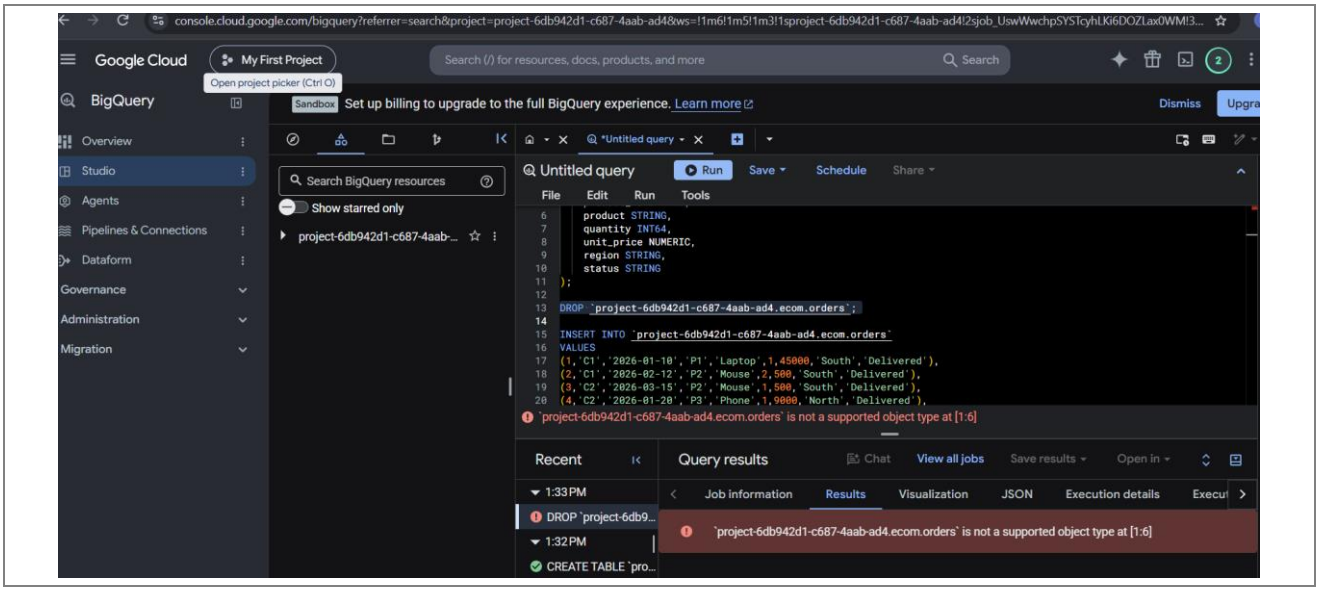
```
SELECT product_id, product,
       SUM(quantity) AS units_sold,
       SUM(quantity * unit_price) AS total_sales
FROM ecom.orders_clean
WHERE UPPER(status) = 'DELIVERED'
  AND order_date BETWEEN '2026-01-01' AND '2026-01-31'
GROUP BY product_id, product
ORDER BY total_sales DESC;
```

RESULT

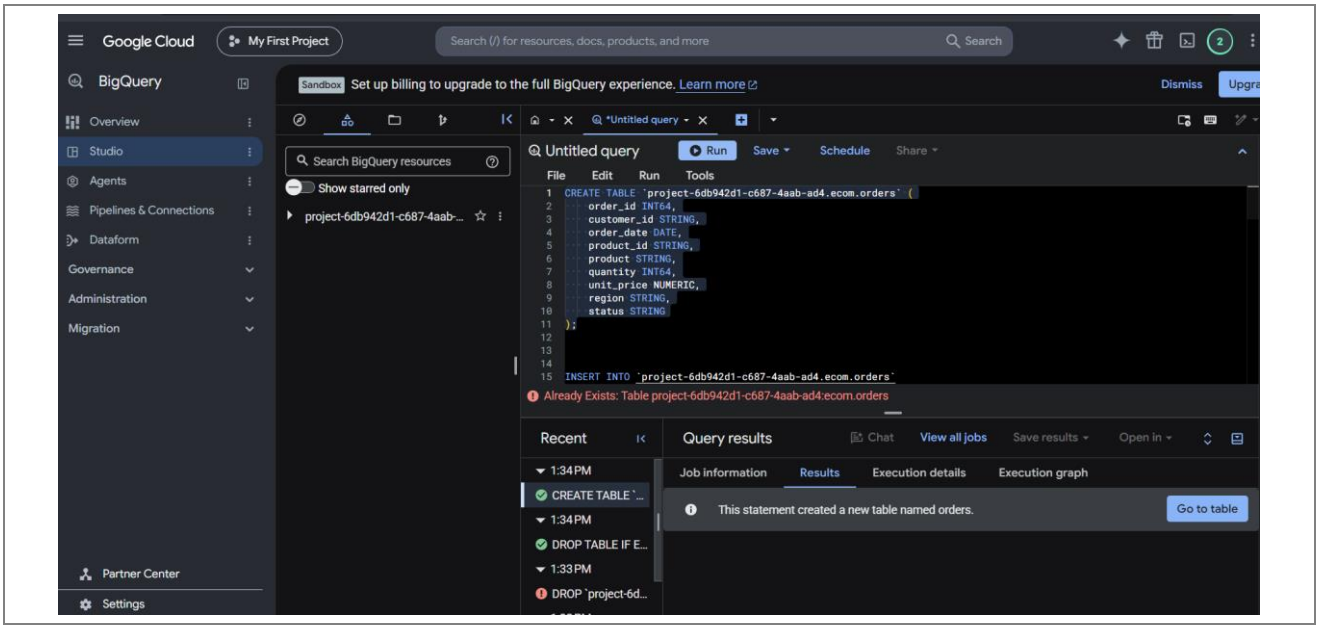
Product ID	Product	Units sold	Total sales
P1	Laptop	1	45,000
P3	Phone	1	9,000

5. Issues Faced and Fixes

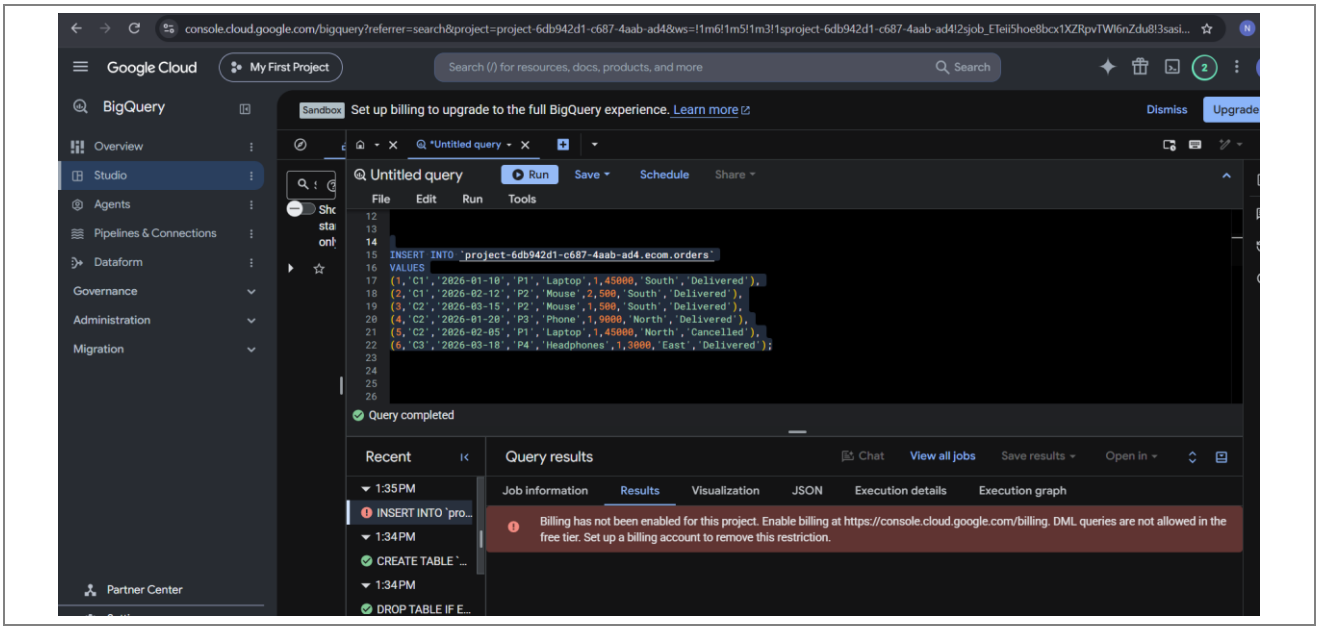
Error message	Cause	Fix
`... is not a supported object type`	DROP used without the object type	Use `DROP TABLE IF EXISTS`
`Already Exists: Table ...ecom.orders`	`CREATE TABLE` run on a table that already exists	Use `CREATE OR REPLACE TABLE`
`Billing has not been enabled ... DML queries are not allowed in the free tier`	`INSERT` is a DML statement and is blocked in the BigQuery Sandbox	Enable billing, load the data by CSV upload, or build the table with `CREATE TABLE ... AS SELECT ... UNION ALL ...`



Screenshot: DROP without TABLE returns "not a supported object type".



Screenshot: "Already Exists" error when re-running CREATE TABLE; the earlier CREATE TABLE succeeded.



Screenshot: INSERT blocked in the free-tier sandbox because billing is not enabled.

6. Key Concepts Practised

- ROW_NUMBER() window function for de-duplication and ranking
- Common table expressions (WITH) for readable multi-step queries
- GROUP BY with HAVING for filtering aggregates
- Date functions: DATE_TRUNC, DATE_SUB, DATE_DIFF and BETWEEN
- Gaps-and-islands technique for consecutive-month streaks
- Business metrics: net revenue, customer lifetime value, average order value and monthly sales

7. Conclusion

All ten tasks were solved with BigQuery SQL. The data was first cleaned by removing duplicate orders, then used to calculate net revenue, rank the top products per region, find inactive and loyal customers, build a customer lifetime value report and summarise monthly and product-wise sales. The queries give the business team a reusable reporting base for the 1,000-orders-per-day dataset.

Prepared by: Naveen Kumar S